

RestEM Implementation - Original SEAL Methodology

This directory contains the **RestEM (Reward-based Self-Training with EM)** implementation following the original SEAL paper methodology.

Two Approaches Available

Option A: PPO Reinforcement Learning (Backed up)

- Located in: `/knowledge-incorporation/src/rl_ppo_backup/`
- Advanced actor-critic implementation with MLX optimization
- Traditional RL with online policy gradients
- TTT server integration for reward evaluation
- **Status:** Fully functional, ready for future development

Option B: RestEM (Original SEAL) (Current)

- Located in: `/knowledge-incorporation/src/restem/`
 - Follows the exact original SEAL paper methodology
 - Behavioral cloning on verified correct solutions
 - **Two implementations:** SQuAD-style QA and ARC Few-Shot
 - **Status:** Complete implementation ready for testing
-

RestEM Methodology Overview

RestEM follows the original SEAL paper with 4 distinct phases:

Implementation A: ARC Few-Shot (Matching Original Paper)

- **Files:** `arc_*.py` files + `arc_restem_pipeline.py`
- **Domain:** Abstract Reasoning Corpus (ARC) tasks
- **Script:** `train_arc_restem_mlx.sh`
- **Matches:** The original SEAL few-shot methodology exactly

Implementation B: SQuAD-style QA (Alternative Domain)

- **Files:** Original `*.py` files + `restem_pipeline.py`
- **Domain:** Question-answering tasks
- **Script:** `train_restem_mlx.sh`
- **Purpose:** Demonstrates RestEM on different task type

Phase 1: Self-Edit Generation

- Generate multiple solution attempts per problem
- Use diverse sampling to create different approaches
- **ARC:** Configuration-based strategies (rotation, color patterns, etc.)
- **QA:** Different reasoning approaches for questions

Phase 2: Test-Time Training (TTT)

- Train individual LoRA adapters for each solution attempt
- Each adapter specializes on one specific solution approach
- **ARC:** `arc_ttt_trainer.py`
- **QA:** `ttt_trainer.py`

Phase 3: Solution Evaluation

- Test each LoRA adapter on the original problem
- Determine which solutions are actually correct
- Assign binary rewards (correct/incorrect)
- **ARC:** `arc_solution_evaluator.py`
- **QA:** `solution_evaluator.py`

Phase 4: RestEM Training

- Train final model using **only verified correct solutions**
- Use behavioral cloning (supervised learning), **not traditional RL**
- Creates the "RL_trained_model" mentioned in original paper
- **ARC:** `arc_restem_trainer.py`
- **QA:** `restem_trainer.py`

How to Use RestEM

Option 1: ARC Few-Shot (Recommended - Matches Original)

```
cd /Users/keenana/Documents/SEAL
bash knowledge-incorporation/scripts/train_arc_restem_mlx.sh
```

Option 2: SQuAD-style QA

```
cd /Users/keenana/Documents/SEAL
bash knowledge-incorporation/scripts/train_restem_mlx.sh
```

ARC Pipeline Usage

```
from src.restem.arc_restem_pipeline import ARCRestEMPipeline
from src.restem.arc_self_edit_generator import load_arc_tasks_from_json

# Load ARC tasks
arc_tasks = load_arc_tasks_from_json("challenges.json", "solutions.json")

# Initialize pipeline
pipeline = ARCRestEMPipeline("mlx-community/Meta-Llama-3-8B-Instruct")
```

```
# Run complete iteration
results = pipeline.run_complete_restem_iteration(
    arc_tasks=arc_tasks[:10], # First 10 tasks
    n_self_edits=15,          # 15 configs per task (matching original)
    ttt_epochs=3,
    restem_epochs=8
)
```

Configuration

ARC RestEM (`train_arc_restem_mlx.sh`):

- **NUM_TASKS**: Number of ARC tasks to process
- **SELF_EDITS_PER_TASK**: Solution attempts per task (15 = original)
- **TTT_EPOCHS**: Training epochs per adapter
- **RESTEM_EPOCHS**: Final model training epochs

SQuAD RestEM (`train_restem_mlx.sh`):

- **NUM_PROMPTS**: Number of QA problems to process
- **SELF_EDITS_PER_PROMPT**: Solution attempts per problem
- **TTT_EPOCHS**: Training epochs per adapter
- **RESTEM_EPOCHS**: Final model training epochs

Expected Output Structure

```
logs/restem_run_1/
├── restem_iteration_1/
│   ├── self_edit_results.json      # Generated solutions
│   ├── ttt_adapters/              # Trained LoRA adapters
│   ├── evaluation_results.json    # Solution correctness results
│   ├── correct_solutions.json     # Filtered correct solutions
│   ├── RL_trained_model/         # Final RestEM model
│   └── restem_iteration.log       # Training logs
└── overall_restem_results.json    # Summary results
```

Switching Between Approaches

To Resume PPO Work (Option A):

```
# Backup current RestEM work
cp -r knowledge-incorporation/src/restem knowledge-
incorporation/src/restem_backup

# Restore PPO implementation
```

```
cp -r knowledge-incorporation/src/rl_ppo_backup/* knowledge-incorporation/src/rl/

# Use PPO training script
bash knowledge-incorporation/scripts/train_rl_mlx_ppo_backup.sh
```

To Resume RestEM Work (Option B):

```
# Already active! Just run:
bash knowledge-incorporation/scripts/train_restem_mlx.sh
```

Key Differences from PPO

Aspect	PPO (Option A)	RestEM (Option B)
Algorithm	Actor-Critic PPO	Behavioral Cloning
Training	Online RL with value functions	Offline supervised learning
Rewards	Continuous from TTT server	Binary correct/incorrect
Data	All generated solutions	Only verified correct solutions
Methodology	Modern RL research	Original SEAL paper

Testing & Validation

1. **Start with small dataset:** Use `NUM_PROMPTS=5` for initial testing
2. **Monitor correctness rate:** Good RestEM should achieve >20% correct solutions
3. **Check adapter training:** Ensure TTT adapters are being created successfully
4. **Validate final model:** Test RestEM model on held-out evaluation set

Next Steps

1. **Run initial RestEM experiment:** Test with small dataset
2. **Evaluate results:** Compare correctness rates and final model performance
3. **Scale up:** Increase to full dataset if initial results are promising
4. **Iterate:** Run multiple RestEM iterations for improved performance

Troubleshooting

- **Memory issues:** Reduce batch sizes in individual components
- **Low correctness:** Check self-edit diversity and evaluation logic
- **TTT failures:** Verify LoRA training is working correctly
- **Import errors:** Ensure MLX environment is properly activated

Date: August 3, 2025

Status: RestEM implementation complete and ready for testing